

S.No: 1	Exp. Name: <i>Write a program in python to calculate and print the Electricity bill of a given customer.</i>	Date: 2025-05-03
----------------	---	-------------------------

Aim:

Write a program in Python to calculate and print the Electricity bill of a given customer. The customer id., name and unit consumed by the user should be taken from the keyboard and display the total amount to pay to the customer.

The charge are as follow :

- For first 50 units Rs. 0.50/unit
- For next 100 units Rs. 0.75/unit
- For next 100 units Rs. 1.20/unit
- For unit above 250 Rs. 1.50/unit

An additional surcharge of 20% is to be added to the bill

Input Format:

Prompt the user to enter the customer ID(integer), customer name(string), units consumed(integer) each on a new line

Output Format:

Print the header for the electricity bill:

Electricity Bill

Print the customer ID:

Customer ID: <customer_id>

Print the customer name:

Customer Name: <customer_name>

Print the units consumed:

Units Consumed: <units>

Print the calculated amount (formatted to two decimal places):

Amount: Rs. <amount>

Print the surcharge (formatted to two decimal places):

Surcharge (20%): Rs. <surcharge_amount>

Print the total amount (formatted to two decimal places):

Total Amount: Rs. <total_amount>

Example

Sample Input:

Enter customer ID: 5184
Enter customer name: Sanjeev
Enter units consumed: 52

Sample Output:

Electricity Bill
Customer ID: 5184
Customer Name: Sanjeev

Units Consumed: 52
Amount: Rs. 26.50
Surcharge (20%): Rs. 5.30
Total Amount: Rs. 31.80

Format the outputs Amount, Surcharge, Total Amount upto two decimals

Source Code:

electricityBill.py

```
id=int(input("Enter customer ID: "))
name=input("Enter customer name: ")
units=int(input("Enter units consumed: "))
print("Electricity Bill")
print("Customer ID:",id)
print("Customer Name:",name)
print("Units Consumed:",units)
amount=0
if units<=50: amount= units*0.50
elif units<=150: amount=50*0.50+(units-50)*0.75
elif units<=250: amount=50*0.50+100*0.75+(units-150)*1.20
else: amount=50*0.50+100*0.75+100*1.20+(units-250)*1.50
surcharge=0.20*amount
print(f'Amount: Rs. {amount:.2f}')
print(f'Surcharge (20%): Rs. {surcharge:.2f}')
print(f'Total Amount: Rs. {(amount+surcharge):.2f}')
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter customer ID:
5184
Enter customer name:
Sanjeev
Enter units consumed:
52
Electricity Bill
Customer ID: 5184
Customer Name: Sanjeev
Units Consumed: 52
Amount: Rs. 26.50
Surcharge (20%): Rs. 5.30

Total Amount: Rs. 31.80

Test Case - 2
User Output
Enter customer ID:
9584
Enter customer name:
Krishna
Enter units consumed:
63
Electricity Bill
Customer ID: 9584
Customer Name: Krishna
Units Consumed: 63
Amount: Rs. 34.75
Surcharge (20%): Rs. 6.95
Total Amount: Rs. 41.70

S.No: 2	Exp. Name: Write a Python program to generate retail shop billing	Date: 2025-05-03
---------	--	------------------

Aim:

Write a Python program to implement a **Retail Shop Billing System**. The program should collect details from the user, calculate the bill based on the items purchased, and display a formatted summary of the purchase.

Input Format:

- **customer_name:** A string that represents the name of the person making the purchase.
- **date_of_purchase:** A string in the format DD/MM/YYYY that indicates the date when the transaction occurred.
- **num_items:** An integer that specifies how many different items the customer is purchasing.
- **item_name:** A string that holds the name of a product being purchased (repeated for each item).
- **quantity:** An integer that represents how many units of the current item were purchased.
- **price_per_unit:** A float that indicates the cost of a single unit of the item.

Output Format:

- Display the **customer_name** and **date_of_purchase**.
- Show a table or list with item-wise details:
- Item Name, Quantity, Price Per Unit, and Total Cost per item.
- At the end, display the "Total Bill Amount" — the sum of all item costs.

Note:

- Refer to the sample test cases for better understanding regarding Input and Output formats.

Source Code:

```
retailBilling.py
```

```

def generate_bill():
    # Get customer details
    customer_name = input("Customer name: ")
    date_of_purchase = input("Date of purchase (DD/MM/YYYY): ")
    num_items = int(input("Number of items purchased: "))

    # Initialize total bill amount

    # Display customer details
    print("-----Retail Shop Billing-----")
    print("Customer Name:", customer_name)
    print("Date of Purchase:", date_of_purchase)

    print("Item-wise Details:")
    print("Item Name\tQuantity\tPrice Per Unit\tTotal Cost")
    print("-" * 50)

    # Iterate over each item
    amount=0
    for i in range(0,num_items):
        print('Item',i+1)
        name=input('Item name: ')
        q=int(input('Quantity: '))
        ppu=float(input('Price per unit: '))
        print(f'{name}\t\t{q}\t\t{ppu}\t\t{(q*ppu):.2f}')
        amount+=q*ppu
        # Calculate total cost for the item

    print(f'Total Bill Amount: {amount:.1f}')
    # Update total bill amount

    # Display item-wise details

    # Display total bill amount

# Call the function to generate the bill
generate_bill()

```

Page No: 5

ID: 2415249

2024-28-CSE1

Guru Nanak Dev Engineering College

Execution Results - All test cases have succeeded!

Test Case - 1
User Output

Customer name:			
Harry			
Date of purchase (DD/MM/YYYY):			
11/05/2023			
Number of items purchased:			
2			
-----Retail Shop Billing-----			
Customer Name: Harry			
Date of Purchase: 11/05/2023			
Item-wise Details:			
Item Name	Quantity	Price Per Unit	Total Cost

Item 1			
Item name:			
Jeans			
Quantity:			
2			
Price per unit:			
399			
Jeans	2	399.0	798.00
Item 2			
Item name:			
Shirt			
Quantity:			
1			
Price per unit:			
299			
Shirt	1	299.0	299.00
Total Bill Amount: 1097.0			

Test Case - 2
User Output
Customer name:
Catherine
Date of purchase (DD/MM/YYYY):
05/07/2023
Number of items purchased:
3
-----Retail Shop Billing-----
Customer Name: Catherine
Date of Purchase: 05/07/2023
Item-wise Details:

Item Name	Quantity	Price Per Unit	Total Cost

Item 1			
Item name:			
Tops			
Quantity:			
3			
Price per unit:			
299			
Tops	3	299.0	897.00
Item 2			
Item name:			
Leggins			
Quantity:			
2			
Price per unit:			
199			
Leggins	2	199.0	398.00
Item 3			
Item name:			
Jeans			
Quantity:			
4			
Price per unit:			
459			
Jeans	4	459.0	1836.00
Total Bill Amount: 3131.0			

Page No: 7

ID: 2415249

2024-28-CSE1

Guru Nanak Dev Engineering College

S.No: 3	Exp. Name: <i>Python program to generate the sine series for a given angle in radians</i>	Date: 2025-05-03
----------------	--	-------------------------

Aim:

Write a Python program to generate the sine series for a given angle in radians. The sine series is computed using the formula:
 $\sin(x) = x - (x^3)/3! + (x^5)/5! - (x^7)/7! + \dots$

Your program should take an angle in radians as input and calculate the sine value using the sine series up to a specified number of terms. The program should print the calculated sine value.

Note: round the final output to 2 decimal places.

Example:

Input:

Angle (in radians): 0.8
Number of terms: 5

Output:

Sine value: 0.72

Source Code:

sineSeries.py

```
# Type Content here....
def fac(x):
    if x==0: return 1
    else: return x*fac(x-1)
angle=float(input('Angle (in radians): '))
n=int(input('Number of terms: '))
s=0
p=0
for i in range(1,2*n,2):
    s+=(-1)**p*(angle**i)/fac(i)
    p+=1
print(f'Sine value: {s:.2f}')
```

Execution Results - All test cases have succeeded!

Test Case - 1

Page No: 8

ID: 2415249

2024-28-CSE1

Guru Nanak Dev Engineering College

User Output
Angle (in radians):
0.8
Number of terms:
5
Sine value: 0.72

Test Case - 2
User Output
Angle (in radians):
10
Number of terms:
4
Sine value: -1307.46

S.No: 4	Exp. Name: <i>Write a Python program to calculate the total weight of the motorbike</i>	Date: 2025-05-03
----------------	--	-------------------------

Aim:

You are designing a program to calculate the weight of a motorbike. The weight of the motorbike is determined by several components: the frame, engine, wheels, and fuel tank. Write a Python program to calculate the total weight of the motorbike based on the user input for the weights of these components.

Sample Test Case:

Input:

Enter the weight of frame (in kg): 35.2
Enter the weight of engine (in kg): 82.6
Enter the weight of each wheel (in kg): 4.5
Enter the weight of fuel tank (in kg): 10.3

Output:

The weight of the motorbike is: 137.1 kg

Note: Assume that the user will always enter valid numeric values for the weights.

Source Code:

weightOfMotorBike.py

```
# Type Content here...
wf=float(input('Enter the weight of frame (in kg): '))
we=float(input('Enter the weight of engine (in kg): '))
ww=float(input('Enter the weight of each wheel (in kg): '))
wt=float(input('Enter the weight of fuel tank (in kg): '))
print('The weight of the motorbike is:',wf+we+ww*2+wt,'kg')
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter the weight of frame (in kg):
35.2
Enter the weight of engine (in kg):
82.6
Enter the weight of each wheel (in kg):

4.5
Enter the weight of fuel tank (in kg):
10.3
The weight of the motorbike is: 137.1 kg

Test Case - 2
User Output
Enter the weight of frame (in kg):
40
Enter the weight of engine (in kg):
90
Enter the weight of each wheel (in kg):
7.8
Enter the weight of fuel tank (in kg):
12
The weight of the motorbike is: 157.6 kg

S.No: 5	Exp. Name: Write a Python program to calculate the weight of a steel bar	Date: 2025-05-03
---------	---	------------------

Aim:

You are tasked with writing a program to calculate the weight of a steel bar. The weight of the steel bar is determined by its length and diameter. The program should prompt the user to enter the length and diameter of the steel bar in meters. Using the given inputs, the program should calculate and display the weight of the steel bar in kilograms.

Write a Python program to implement the above requirements. Ensure that the program includes a function **calculate_steel_bar_weight()** which takes the *length* and *diameter* as input parameters and returns the weight of the steel bar.

Note:

1. Assume that the density of steel as 7850 kg/m^3 and the steel bar is cylindrical in shape.
2. Round the final result to three decimal places
3. Take the pi value as **3.14159**.

Sample Test Case:

Input:

Enter the length of the steel bar (in meters): 5

Enter the diameter of the steel bar (in meters): 1

Output:

The weight of the steel bar is: 30826.852 kg

Source Code:

```
WeightOfSteelBar.py
```

```
# Type Content here...
def calculate_steel_bar_weight(l,d):
    volume=3.14159*(d/2)**2*l
    mass=7850*volume
    return mass

# Prompt the user for input
length = float(input("Enter the length of the steel bar (in
meters): "))
diameter = float(input("Enter the diameter of the steel bar (in
meters): "))

# Calculate the weight of the steel bar
steel_bar_weight = calculate_steel_bar_weight(length, diameter)

# Print the result
print("The weight of the steel bar is:",
round(steel_bar_weight,3), "kg")
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter the length of the steel bar (in meters):
5
Enter the diameter of the steel bar (in meters):
1
The weight of the steel bar is: 30826.852 kg
Test Case - 2

User Output
Enter the length of the steel bar (in meters):
10
Enter the diameter of the steel bar (in meters):
2
The weight of the steel bar is: 246614.815 kg

S.No: 6	Exp. Name: <i>Exchange the values of two variables without using a temporary variable</i>	Date: 2025-03-31
----------------	--	-------------------------

Aim:

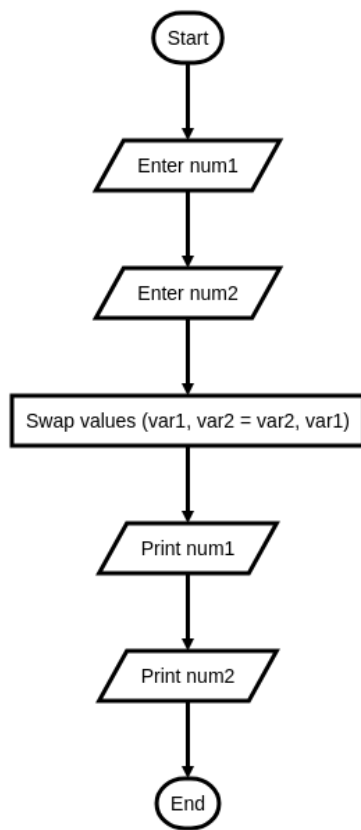
Aim:

Write a Python program to exchange the values of two variables, num1 and num2, without using a temporary (third) variable.

Algorithm:

1. Start
2. Take input from the user for the first number and store it in the variable num1.
3. Take input from the user for the second number and store it in the variable num2.
4. Swap the values of num1 and num2 using tuple unpacking: num1, num2 = num2, num1.
5. Print the exchanged values of num1 and num2 with appropriate labels.

Flowchart:



Input Format:

- The user will be prompted to enter two numbers (**num1** and **num2**) in different lines

Output Format:

- After exchanging the values of **num1** and **num2**, the program will output the new values.
- The output will include the labels and the exchanged values of **num1** and **num2**.

Note:

- Please map your input and output according to the sample visible test cases.

Source Code:

exchange.py

```
#Write your code here
n1=int(input('num1: '))
n2=int(input('num2: '))
n1=n1+n2
n2=n1-n2
n1=n1-n2
print('After exchanging values:')
print('num1:',n1)
print('num2:',n2)
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
num1:
9
num2:
8
After exchanging values:
num1: 8
num2: 9

Test Case - 2
User Output
num1:
65
num2:
27
After exchanging values:
num1: 27
num2: 65

Page No: 17

ID: 2415249

2024-28-CSE1

Guru Nanak Dev Engineering College

S.No: 7	Exp. Name: <i>Circulate the values of n variables</i>	Date: 2025-04-01
----------------	--	-------------------------

Aim:

Write a Python program that takes n variable values as input (separated by spaces) and circulates these values to the right by one position. This means the last value moves to the first position, the second-to-last value moves to the second, and so on. The program should then display the **circulated values**, each on a new line, prefixed by the text "**After circulating values:**".

Input format:

- The user will provide a single line of input containing n values separated by spaces.

Output format:

- After circulating values:** The program should output the right-circulated values, with each value printed on a new line.

Source Code:

circulate.py

```
#Type Content here...
n=input()
print('After circulating values:')
l=n.split()
for i in range(-1,len(l)-1):
    print(l[i])
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
8 9 15 60
After circulating values:
60
8
9
15
Test Case - 2

User Output
30 100 98 65.235
After circulating values:
65.235
30
100
98

S.No: 8	Exp. Name: Distance between two points	Date: 2025-04-01
---------	---	------------------

Aim:

Write a Python program to calculate the **Euclidean distance** between two points in a two-dimensional Cartesian coordinate system. The program will take the coordinates of the two points, (x_1, y_1) and (x_2, y_2) , as input in a specific format. It will then calculate the distance using the distance formula and display the result, rounded to two decimal places.

Input format:

- The user will provide a single line of input containing four integer values separated by spaces, representing the x and y coordinates of the two points in the order: $x_1\ y_1\ x_2\ y_2$.

Output format:

- The program should output the calculated distance as a floating-point number, rounded to two decimal places.

Note:

- Restrict the output to two decimal places. Use math module in python
- Distance Between Two Points: $\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$.

Source Code:

```
distance.py
#Type Content here...
d=input()
l=list(map(float,d.split(' ')))
print(f"{{{(l[2]-l[0])**2+(l[3]-l[1])**2}**(1/2):.2f}}")
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
3 0 0 4
5.00

Test Case - 2
User Output

1 2.5 4 6
4.61

S.No: 9	Exp. Name: <i>Print even number series</i>	Date: 2025-03-31
----------------	--	-------------------------

Aim:

Write a Python program to generate and print a series of the first N even numbers.

Input format:

- An integer N represents the number of terms in the series.

Output format:

- Print the first N even numbers, separated by a space.

Source Code:

```

series.py

# Write your code here...
n=int(input())
for i in range(2,2*n+1,2):
    print(i,end=' ')

```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
5
2 4 6 8 10

Test Case - 2
User Output
10
2 4 6 8 10 12 14 16 18 20

S.No: 10	Exp. Name: <i>Print number pattern</i>	Date: 2025-03-31
-----------------	---	-------------------------

Aim:

Write a Python program to print a number pattern as shown in the example.

Input format:

- The input should be a number, an integer n represents the number of rows in the pattern.

Output format:

- Print a number pattern with n rows as described in the example.

Example:

Input: 5

Output:

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Source Code:

numberPattern.py

```
#Type Content here...
n=int(input())
for i in range(1,n+1):
    for j in range(1,i+1):
        if(j==i):
            print(j)
        else:
            print(j,end=' ')
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
8
1

1 2
1 2 3
1 2 3 4
1 2 3 4 5
1 2 3 4 5 6
1 2 3 4 5 6 7
1 2 3 4 5 6 7 8

Test Case - 2
User Output
5
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5

S.No: 11	Exp. Name: <i>Print pyramid pattern</i>	Date: 2025-03-31
-----------------	--	-------------------------

Aim:

Write a Python program to print a **pyramid pattern** as shown in the sample test cases.

Input format:

- The input should be the number of rows to print an integer n represents the number of rows in the pyramid.

Output format:

- Print a pyramid pattern with n rows as described in the example.

Example:

Input: 4

Output:

```
*
***
*****
*****
```

Source Code:

pyramid.py

```
#Type Content here...
n=int(input())
for i in range(1,n+1):
    print(' '*(n-i)+'*'(2*i-1))
    """    for j in range(i,n):
            print(' ',end='')
            for k in range(1,2*i):
                print('*',end='')
            print()"""
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
4

*

Test Case - 2
User Output
5
*

S.No: 12	Exp. Name: <i>Library Items Using Lists</i>	Date: 2025-05-03
-----------------	--	-------------------------

Aim:

You are required to implement a simple library management application that allows users to manage a collection of items (such as books, magazines, etc.). The application should provide options to display the current library inventory, add new items, remove items, and exit the application.

Implement the following functions:

- **display_library(library)**: This function takes a list of items and prints each item in the library.
- **add_item(library, item)**: This function takes the library list and an item to add, appending the item to the library.
- **remove_item(library, item)**: This function takes the library list and an item to remove. If the item exists, it removes it, otherwise it prints a "not found" message.

Instructions:

- Initialize an empty library list.
- Use a loop to continuously display a menu that allows the user to:
 - Display the library inventory
 - Add a new item to the library
 - Remove an existing item from the library
 - Quit the application

Input Format:

- The application should prompt the user for their choice of action (1 to 4). If the user enters an invalid choice, print: "Invalid choice"
- If the user chooses to add or remove an item, the application should prompt for the item name.

Output Format:

The program should print appropriate messages based on user actions, such as:

- Displaying the current library inventory.
- Confirming when an item has been added or removed.
- Indicating if an item to remove was not found.
- Exiting the application.

Note :

- Refer to sample test cases for better understanding of input and output format.

Source Code:

```
libraryApplication.py
```

```

def display_library(library):
    if library==None:
        print('Library is empty')
    else:
        print('Library Inventory:')
        for item in library:
            print(item)
        #write your code here

def add_item(library, item):
    library.append(item)
    print('added to the library')
    #write your code here

def remove_item(library, item):
    if item in library:
        library.remove(item)
        print('removed farom the library')
    else:
        print('not found')
        #write your code here

# Initialize an empty library list
library=[]

while True:
    print("Library Application")
    print("1.Display Library")
    print("2.Add Item")
    print("3.Remove Item")
    print("4.Quit")

    choice = input("Enter your choice:")

    if choice == "1":
        display_library(library)
    elif choice == "2":
        item = input("item to add:")
        add_item(library, item)
    elif choice == "3":
        item = input("item to remove:")
        remove_item(library, item)
    elif choice == "4":
        print("Exiting the library application")
        break
    else:
        print("Invalid choice")

```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Library Application
1.Display Library
2.Add Item
3.Remove Item
4.Quit
Enter your choice:
5
Invalid choice
Library Application
1.Display Library
2.Add Item
3.Remove Item
4.Quit
Enter your choice:
4
Exiting the library application

Test Case - 2
User Output
Library Application
1.Display Library
2.Add Item
3.Remove Item
4.Quit
Enter your choice:
2
item to add:
book1
added to the library
Library Application
1.Display Library
2.Add Item
3.Remove Item
4.Quit
Enter your choice:
2
item to add:
book2

Page No: 29

ID: 2415249

2024-28-CSE1

Guru Nanak Dev Engineering College

added to the library
Library Application
1.Display Library
2.Add Item
3.Remove Item
4.Quit
Enter your choice:
3
item to remove:
book9
not found
Library Application
1.Display Library
2.Add Item
3.Remove Item
4.Quit
Enter your choice:
3
item to remove:
book1
removed farom the library
Library Application
1.Display Library
2.Add Item
3.Remove Item
4.Quit
Enter your choice:
4
Exiting the library application

S.No: 13	Exp. Name: <i>Implement application of Items present in a library using Tuple data structure</i>	Date: 2025-05-03
-----------------	---	-------------------------

Aim:

Write a Python program to implement application of Items present in a library using Tuple data structure.

Source Code:

```
libraryUsingTuple.py
```

Page No: 31

ID: 2415249

2024-28-CSE1

Guru Nanak Dev Engineering College

```

library = ()
def display(library):
    if library==None:
        print('Library is empty')
    else:
        print('Library Inventory:')
        for item in library:
            print(item)

def add(library):
    item=input('item to add:')
    lib=library+(item,)
    print('added to the library')
    return lib

def remove(library):
    item=input('item to remove:')
    if item in library:
        print('removed from the library')
        return
    library[0:library.index(item)+library[(library.index(item)+1):]
    else:
        print('not found')
        return library

while True:
    print("Library Application")
    print("1.Display Library")
    print("2.Add Item")
    print("3.Remove Item")
    print("4.Quit")
    c=int(input('Enter your choice:'))
    if c==1:
        display(library)
    elif c==2:
        library=add(library)
    elif c==3:
        library=remove(library)
    elif c==4:
        print('Exiting the library application')
        break
    else:
        print('Invalid choice')
# Write your code here ...

```

Page No: 32

ID: 2415249

2024-28-CSE1

Guru Nanak Dev Engineering College

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Library Application
1.Display Library
2.Add Item
3.Remove Item
4.Quit
Enter your choice:
2
item to add:
book1
added to the library
Library Application
1.Display Library
2.Add Item
3.Remove Item
4.Quit
Enter your choice:
2
item to add:
book2
added to the library
Library Application
1.Display Library
2.Add Item
3.Remove Item
4.Quit
Enter your choice:
2
item to add:
book9
added to the library
Library Application
1.Display Library
2.Add Item
3.Remove Item
4.Quit
Enter your choice:
1
Library Inventory:
book1
book2
book9

Library Application
1.Display Library
2.Add Item
3.Remove Item
4.Quit
Enter your choice:
3
item to remove:
book16
not found
Library Application
1.Display Library
2.Add Item
3.Remove Item
4.Quit
Enter your choice:
3
item to remove:
book1
removed from the library
Library Application
1.Display Library
2.Add Item
3.Remove Item
4.Quit
Enter your choice:
1
Library Inventory:
book2
book9
Library Application
1.Display Library
2.Add Item
3.Remove Item
4.Quit
Enter your choice:
4
Exiting the library application

Test Case - 2
User Output
Library Application
1.Display Library

2.Add Item
3.Remove Item
4.Quit
Enter your choice:
2
item to add:
book16
added to the library
Library Application
1.Display Library
2.Add Item
3.Remove Item
4.Quit
Enter your choice:
5
Invalid choice
Library Application
1.Display Library
2.Add Item
3.Remove Item
4.Quit
Enter your choice:
2
item to add:
book8
added to the library
Library Application
1.Display Library
2.Add Item
3.Remove Item
4.Quit
Enter your choice:
3
item to remove:
book16
removed from the library
Library Application
1.Display Library
2.Add Item
3.Remove Item
4.Quit
Enter your choice:
3
item to remove:

jack
not found
Library Application
1.Display Library
2.Add Item
3.Remove Item
4.Quit
Enter your choice:
1
Library Inventory:
book8
Library Application
1.Display Library
2.Add Item
3.Remove Item
4.Quit
Enter your choice:
4
Exiting the library application

S.No: 14	Exp. Name: <i>Implement Application of Elements of Civil Structure using Sets</i>	Date: 2025-05-03
-----------------	--	-------------------------

Aim:

You are assigned to build a **Civil Structure Element Manager** in Python. This program will allow users to manage a collection of structural elements (like beams, columns, slabs, etc.) used in a civil construction project. The elements will be stored in a **set**, which allows for fast addition and removal while automatically preventing duplicates.

Your program should provide the following operations:

- Add a new element to the structure
- Remove an existing element from the structure
- Display all current elements in the structure
- Quit the application

To achieve this, you will implement the following functions:

- **add_element**: Adds an element to the set and displays a confirmation message.
- **remove_element**: Removes the element from the set if it exists; otherwise, prints a message that it wasn't found.
- **display_elements**: Prints all current elements in the structure in sorted order.

The application should display a menu and keep running in a loop until the user selects the **Quit** option.

Input Format:

- The user interacts with the program through a **menu**.
- For "**Add Element**", the user inputs the name of the element to add.
- For "**Remove Element**", the user inputs the name of the element to remove.

Output Format:

- When the user selects "**Display Elements**", show the list of elements with the heading: "**Elements in the Civil Structure**", followed by each element on a new line.
- When an element is added, display: "**added to the civil structure**".
- When an element is removed, display: "**removed from the civil structure**".
 - If the element does not exist, display: "**not found**".
- When the user selects "**Quit**", display: "**Exiting the civil structure application**".

Note:

- Refer to the sample test cases for a better understanding of the input and output format.

Source Code:

civilStructureSets.py

```
# Write your code here ...
def display_elements(elements_set):
    if elements_set==None:
        print('No elements in the structure')
    else:
        print('Elements in the Civil Structure:')
        for i in elements_set:
            print(i)
def add_element(elements_set,item):
    elements_set.add(item)
    print('added to the civil structure')
def remove_element(elements_set,item):
    if item in elements_set:
        elements_set.remove(item)
        print('removed from the civil structure')
    else:
        print('not found')

# Initialize an empty set for civil structure elements
elements_set = set()

while True:
    print("Civil Structure Application")
    print("1.Add Element")
    print("2.Remove Element")
    print("3.Display Elements")
    print("4.Quit")

    choice = input("Enter your choice:")

    if choice == "1":
        element = input("element to add:")
        add_element(elements_set, element)
    elif choice == "2":
        element = input("element to remove:")
        remove_element(elements_set, element)
    elif choice == "3":
        display_elements(sorted(elements_set))
    elif choice == "4":
        print("Exiting the civil structure application")
        break
    else:
        print("Invalid choice")
```

Page No: 38

ID: 2415249

2024-28-CSE1

Guru Nanak Dev Engineering College

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Civil Structure Application
1.Add Element
2.Remove Element
3.Display Elements
4.Quit
Enter your choice:
1
element to add:
floor
added to the civil structure
Civil Structure Application
1.Add Element
2.Remove Element
3.Display Elements
4.Quit
Enter your choice:
1
element to add:
roof
added to the civil structure
Civil Structure Application
1.Add Element
2.Remove Element
3.Display Elements
4.Quit
Enter your choice:
1
element to add:
blueprint
added to the civil structure
Civil Structure Application
1.Add Element
2.Remove Element
3.Display Elements
4.Quit
Enter your choice:
1
element to add:
blueprint

Page No: 39

ID: 2415249

2024-28-CSE1

Guru Nanak Dev Engineering College

added to the civil structure
Civil Structure Application
1.Add Element
2.Remove Element
3.Display Elements
4.Quit
Enter your choice:
3
Elements in the Civil Structure:
blueprint
floor
roof
Civil Structure Application
1.Add Element
2.Remove Element
3.Display Elements
4.Quit
Enter your choice:
5
Invalid choice
Civil Structure Application
1.Add Element
2.Remove Element
3.Display Elements
4.Quit
Enter your choice:
2
element to remove:
roof
removed from the civil structure
Civil Structure Application
1.Add Element
2.Remove Element
3.Display Elements
4.Quit
Enter your choice:
1
element to add:
beam
added to the civil structure
Civil Structure Application
1.Add Element
2.Remove Element
3.Display Elements
4.Quit

Enter your choice:
3
Elements in the Civil Structure:
beam
blueprint
floor
Civil Structure Application
1.Add Element
2.Remove Element
3.Display Elements
4.Quit
Enter your choice:
4
Exiting the civil structure application

Test Case - 2
User Output
Civil Structure Application
1.Add Element
2.Remove Element
3.Display Elements
4.Quit
Enter your choice:
5
Invalid choice
Civil Structure Application
1.Add Element
2.Remove Element
3.Display Elements
4.Quit
Enter your choice:
4
Exiting the civil structure application

S.No: 15	Exp. Name: <i>Implement application of components of automobiles using Dictionaries</i>	Date: 2025-05-03
-----------------	--	-------------------------

Aim:

You are required to build an **Automobile Components Manager** in Python. This program allows users to maintain a record of automobiles and their corresponding components using a **dictionary**, where the **key** is the automobile name and the **value** is its components.

The system should provide a menu with the following options:

- Display all automobile-component pairs.
- Add a new automobile with its components.
- Remove an automobile and its associated components.
- Quit the application.

To implement this, define the following functions:

- **display_components()**: Displays all automobile-component pairs in the format **automobile:component**
- **add_component()**: Adds or updates a component for a given automobile and prints a confirmation message.
- **remove_component()**: Removes a given automobile from the record if it exists; otherwise, prints a message that it wasn't found.

The application should continue to prompt the user for choices until they select the **Quit** option.

Input Format:

- The application should prompt the user for their choice of action (1 to 4).
- For "**Add Component**", the user inputs the automobile name and the component to add.
- For "**Remove Component**", the user inputs the name of the automobile to remove.

Output Format:

- When the user selects "**Display Components**", show all automobile-component pairs under the heading: "**Components of Automobiles:**" followed by each pair in a new line in the format: **automobile:component**.
- When an automobile is added, display "**added**".
- When an automobile is removed, display "**removed**".
 - If the automobile is not found, display "**not found**".
- When the user selects "**Quit**", display "**Exiting the automobile components application**".

Note:

- Refer to the sample test cases to understand the input and output format.

Source Code:

```
automobiles.py
```

Page No: 43

ID: 2415249

2024-28-CSE1

Guru Nanak Dev Engineering College

```

#Type Content here...
def display_components(automobile_components):
    if automobile_components==None:
        print('No components in the system')
    else:
        print('Components of Automobiles:')
        for k,v in automobile_components.items():
            print(f'{k}:{v}')
def add_component(automobile_components,key,value):
    automobile_components[key]=value
    print('added')
def remove_component(automobile_components,item):
    if item in automobile_components:
        del automobile_components[item]
        print('removed')
    else:
        print('not found')

# Initialize an empty dictionary for automobile components
automobile_components = {}

while True:
    print("Automobile Components Application")
    print("1.Display Components")
    print("2.Add Component")
    print("3.Remove Component")
    print("4.Quit")

    choice = input("Enter your choice:")

    if choice == "1":
        display_components(automobile_components)
    elif choice == "2":
        automobile = input("name of the automobile:")
        component = input("component to add:")
        add_component(automobile_components, automobile,
component)
    elif choice == "3":
        automobile = input("name of the automobile:")
        remove_component(automobile_components, automobile)
    elif choice == "4":
        print("Exiting the automobile components application")
        break
    else:
        print("Invalid choice")

```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Automobile Components Application
1.Display Components
2.Add Component
3.Remove Component
4.Quit
Enter your choice:
2
name of the automobile:
ford
component to add:
engineer
added
Automobile Components Application
1.Display Components
2.Add Component
3.Remove Component
4.Quit
Enter your choice:
2
name of the automobile:
buik
component to add:
glass
added
Automobile Components Application
1.Display Components
2.Add Component
3.Remove Component
4.Quit
Enter your choice:
1
Components of Automobiles:
ford:engineer
buik:glass
Automobile Components Application
1.Display Components
2.Add Component
3.Remove Component
4.Quit

Page No: 45

ID: 2415249

2024-28-CSE1

Guru Nanak Dev Engineering College

Enter your choice:
5
Invalid choice
Automobile Components Application
1.Display Components
2.Add Component
3.Remove Component
4.Quit
Enter your choice:
3
name of the automobile:
ford
removed
Automobile Components Application
1.Display Components
2.Add Component
3.Remove Component
4.Quit
Enter your choice:
1
Components of Automobiles:
buik:glass
Automobile Components Application
1.Display Components
2.Add Component
3.Remove Component
4.Quit
Enter your choice:
4
Exiting the automobile components application

Test Case - 2
User Output
Automobile Components Application
1.Display Components
2.Add Component
3.Remove Component
4.Quit
Enter your choice:
5
Invalid choice
Automobile Components Application
1.Display Components

2.Add Component
3.Remove Component
4.Quit
Enter your choice:
3
name of the automobile:
jaguar
not found
Automobile Components Application
1.Display Components
2.Add Component
3.Remove Component
4.Quit
Enter your choice:
4
Exiting the automobile components application

S.No: 16	Exp. Name: <i>Factorial of a number</i>	Date: 2025-04-29
-----------------	--	-------------------------

Aim:

Write a Python program to find the factorial of a number using functions.

Input Format:

- The input should be a non-negative integer.

Output Format:

- The output should be the factorial of that number.

Source Code:

```
factorial.py

def factorial(n):
    #Wrtie your code here...
    if n==0: return 1
    else:return n*factorial(n-1)

# Taking input from the user
num = int(input())

# Calculating the factorial
print(factorial(num))
```

Execution Results - All test cases have succeeded!

Test Case - 1	
User Output	
5	
120	

Test Case - 2	
User Output	
10	
3628800	

S.No: 17	Exp. Name: <i>Largest number in a list</i>	Date: 2025-04-29
-----------------	---	-------------------------

Aim:

Write a Python program to find the largest number in a list using functions.

Input Format:

- The input should be a list of numbers separated by spaces.

Output Format:

- The output should be the largest number in that list.

Source Code:

```
largest.py

def find_largest(numbers):
    # Write your code here..
    return max(numbers)

# Taking input from the user
input_str = input()
numbers = (int(x) for x in input_str.split())

# Finding the largest number
largest_number = find_largest(numbers)

print(largest_number)
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
2 5 9 3 5 8
9

Test Case - 2
User Output
58 65 25 4 5 30

S.No: 18	Exp. Name: <i>Area of a rectangle</i>	Date: 2025-04-29
-----------------	--	-------------------------

Aim:

Imagine you're tasked with developing a program to assist a construction project manager. They need a tool to quickly calculate the area of various rectangular sections of land before planning further construction details. Your task is to create a Python program that accurately computes the area of a rectangle based on user-provided dimensions.

Input Format:

- **"length:"** - A floating-point number representing the length of the rectangular section of land.
- **"width:"** - A floating-point number representing the width of the rectangular section of land.

Output Format:

- **"Area:"** - Displays the calculated area of the rectangular section of land.

Source Code:

rectangle.py

```
def rectangle_area(length, width):
    # Complete the function
    return length*width

# Taking input from the user
length,width=float(input('length:')),float(input('width:'))

# Calculating the area of the rectangle
area = rectangle_area(length, width)
print(f"Area:{area}")
#Print the area
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
length:
15
width:

10
Area:150.0

Test Case - 2
User Output
length:
12.56
width:
5.654
Area:71.01424

S.No: 19	Exp. Name: <i>Reverse of a string</i>	Date: 2025-05-02
-----------------	--	-------------------------

Aim:

Write a Python program to find the reverse of a string.

Input Format:

- The input should be a string.

Output Format:

- The output should be the reverse of a string.

Source Code:

```
reverseString.py

#Type Content here...
s=input()
print(s[::-1])
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
CodeTantra
artnaTedoC

Test Case - 2
User Output
Programming
gnimmargorP

S.No: 20	Exp. Name: <i>Palindrome checker</i>	Date: 2025-05-02
-----------------	---	-------------------------

Aim:

Write a Python program to find whether a given string is a palindrome or not.

Note: A palindrome is a string that reads the same backwards as forward. For example, "racecar" is a palindrome because it reads the same in both directions, while "hello" is not.

Input Format:

- The input should be a string.

Output Format:

- If the given string is a palindrome, print "**palindrome**".
- Otherwise, print "**not a palindrome**".

Source Code:

```
palindrome.py
# Type Content here...
s=input()
r=s[::-1]
if r==s:
    print('palindrome')
else:
    print('not a palindrome')
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
mam
palindrome

Test Case - 2
User Output
sir
not a palindrome

S.No: 21	Exp. Name: <i>Character count of a string</i>	Date: 2025-05-02
-----------------	--	-------------------------

Aim:

Write a Python program to find the character count of a string.

Input Format:

- **string:** This line will prompt the user to input a string.

Output Format:

- **Character count:** It should display the character and its count in a new line for each character.

Source Code:

charCount.py

```
#Type Content here...
s=input('string:')
l=[]
print('Character count:')
for i in s:
    if i not in l:
        l.append(i)
        print(f'{i}: {s.count(i)}')
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
string:
CodeTantra
Character count:
C: 1
O: 1
d: 1
e: 1
T: 1
a: 2
n: 1
t: 1
r: 1

Test Case - 2
User Output
string:
aaaaaabbbbbccccccddddd
Character count:
a: 6
b: 6
c: 6
d: 7

S.No: 22	Exp. Name: <i>Replace a character in the string</i>	Date: 2025-05-02
-----------------	--	-------------------------

Aim:

Write a Python program to replace a character in the string.

Input Format:

- The first line of input should be a string.
- The second line of input contains a character to be replaced.
- The third line of input contains a new character to replace the existing character.

Output Format:

- The output should be the modified string.

Source Code:

replaceChar.py

```
# Type Content here...
a,b,c=input(),input(),input()
d=a.replace(b,c)
print(d)
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Codetantra
t
T
CodeTanTra

Test Case - 2
User Output
abbbbbbbccccccddddd
b
F
aFFFFFFccccccddddd

S.No: 23	Exp. Name: <i>Copy contents from one file to another</i>	Date: 2025-05-05
-----------------	---	-------------------------

Aim:

Write a Python program to copy the contents from one file to another.

Input Format:

- The first line of input should be the source filename.
- The second line of input should be the target filename.

Output Format:

- The output should be "**Contents copied**", which will be displayed after copying the content of the file.

Note:

- Consider file1.txt, file2.txt as the input file names.

Source Code:

copyFiles.py

```
import os
def copy_file(source_file, target_file):
    with open(source_file, 'r') as src:
        content=src.read()
    with open(target_file, 'w') as tgt:
        tgt.write(content)
    #write your function code here

source_file = input()
target_file = input()

#write your code here
copy_file(source_file, target_file)
if os.path.getsize(target_file) == 0:
    print("Contents not copied")
else:
    print("Contents copied")
```

file2.txt

Hello there lets learn python programming

input1.txt

sun and the moon

input2.txt

dgfsfdgdfhrsgfdh
sfdgtedgrfdh

input3.txt

asdgfsgfdgfdgfsfddfg

file1.txt

CodeTantra
Start your coding in 60 mins

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
file1.txt
output.txt
Contents copied

Test Case - 2
User Output
file2.txt
output.txt
Contents copied

S.No: 24	Exp. Name: <i>Count the number of words in a file</i>	Date: 2025-05-05
-----------------	--	-------------------------

Aim:

Write and execute a Python program to count the number of words in a file.

Input format:

- The input should be filename.

Output format:

- The output should be number of words in that file.

Note: Consider file1.txt, file2.txt as the input file names.

Aim:

- To create a Python program that counts the number of words in a file.

Algorithm:

- Step 1: Define a function named count_words(filename) that takes a filename as input.
- Step 2: Open the file specified by filename in read mode using a context manager (with statement).
- Step 3: Read the content of the file and split it into words using the split() method.
- Step 4: Return the number of words.
- Step 5: Handle the FileNotFoundError exception by printing "File not found" and returning -1.
- Step 6: Accept a filename input from the user.
- Step 7: Call the count_words() function with the input filename.
- Step 8: Print the word count if it's greater than or equal to 0.
- Step 9: End the program.

Source Code:

```
wordCount.py
```

```
# Type Content here...
def count_words(filename):
    try:
        with open(filename,'r') as file:
            content=file.read()
            words=content.split()
            return len(words)
    except FileNotFoundError:
        print('File not found')
        return -1

filename=input()
word_count=count_words(filename)
print(word_count)
```

file1.txt

CodeTantra Start your coding in 60 mins

file2.txt

hello there lets start coding in python programming language

input1.txt

sun and moon

input2.txt

where there is a will there is a way

input3.txt

count the number of words in this file

Execution Results - All test cases have succeeded!

Test Case - 1
User Output

file1.txt
7

Test Case - 2
User Output
file2.txt
9

S.No: 25	Exp. Name: <i>Longest word in a file</i>	Date: 2025-05-05
-----------------	---	-------------------------

Aim:

Write a Python program to find the longest word in a file.

Input Format:

- The input should be a filename.

Output Format:

- The output should be the longest word.

Note: Consider file1.txt, file2.txt as the input file names.

Source Code:

longestWord.py

```
#Type Content here...
def max_word(filename):
    with open(filename,'r') as file:
        content=file.read()
        words=content.split()
        return max(words,key=len)
filename=input()
longest_word=max_word(filename)
print(longest_word)
```

file1.txt

CodeTantra lets start coding in 60 mins

file2.txt

hello there lets start coding in python programming language

input1.txt

sun and moon

input2.txt

Page No: 63

ID: 2415249

2024-28-CSE1

Guru Nanak Dev Engineering College

hello there!!

input3.txt

Compilation Errors

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
file2.txt
programming

Test Case - 2
User Output
file1.txt
CodeTantra

S.No: 26	Exp. Name: Program to divide by zero error using exception handling	Date: 2025-05-03
----------	--	------------------

Aim:

Write a Python program that divides two numbers and demonstrates how exception handling can be used to catch and handle a divide by zero error. If the user attempts to divide by zero, the program should handle the exception and display an appropriate error message.

Input Format:

- The input should consist of two integers separated by a space, where the first integer is the numerator and the second is the denominator.

Output Format:

- If the division is successful, the output should be the result of the division.
- If a divide by zero error occurs, the output should be: **Error: Division by zero is not allowed.**

Source Code:

```
zeroDivisionError.py

try:
    a,b=input().split()
    print(int(a)/int(b))
    #Write your code here...

except ZeroDivisionError:
    print("Error:Division by zero is not allowed")
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
9 0
Error:Division by zero is not allowed

Test Case - 2
User Output
10 2

5.0

ID: 2415249 Page No: 66

2024-28-CSE1
Guru Nanak Dev Engineering College

S.No: 27	Exp. Name: Validate voter's age using exception handling	Date: 2025-05-04
----------	---	------------------

Aim:

Write a Python program that validates a voter's age using exception handling. The program should check if the entered age is 18 or older and display a message accordingly. If the age is below 18, the program should raise an exception and display an error message.

Input Format:

- **age:** Prompt the user to input their age (as an integer).

Output Format:

- If the entered age is 18 or older, the program should output: "**You are eligible to vote**"
- If the entered age is less than 18, an error message is displayed: "**Error:You must be at least 18 years old to vote**".

Source Code:

```
voterException.py

def validate_age(age):
    if age<18:
        raise ValueError
    print('You are eligible to vote')
    #write your function code here

try:
    a=int(input('age:'))
    validate_age(a)#write try block code

except ValueError:
    print('Error:You must be at least 18 years old to vote')
    # write your code
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
age:
15
Error:You must be at least 18 years old to vote

Test Case - 2
User Output
age:
20
You are eligible to vote

S.No: 28	Exp. Name: Validate student mark range using exception handling	Date: 2025-05-04
----------	--	------------------

Aim:

Write a Python program to validate the range of a student's marks using exception handling. The program should check if the entered marks are within the valid range (0 to 100). If the marks are within this range, the program should display a confirmation message. If the marks are out of range, an exception should be raised and an appropriate error message should be displayed.

Input Format:

- **marks:** Prompt the user to input the marks of a student (as an integer).

Output Format:

- If the entered marks are in the range of 0 to 100(inclusive of both), then print **"Marks are valid"**.
- If the entered marks are less than 0 or greater than 100, print the error message: **"Error: Marks must be in the range of 0 to 100"**.

Source Code:

```
marksException.py

# write your code here
def validity(marks):
    if marks<0 or marks>100:
        raise ValueError
    print("Marks are valid")
try:
    m=int(input('marks:'))
    validity(m)
    # write your code here

except ValueError :
    print('Error:Marks must be in the range of 0 to 100')
    # write your code here
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
marks:

105
Error:Marks must be in the range of 0 to 100

Test Case - 2
User Output
marks:
95
Marks are valid

S.No: 29	Exp. Name: NumPy	Date: 2025-05-05
-----------------	-------------------------	-------------------------

Aim:

Write a NumPy program to calculate the difference between the maximum and the minimum values of a given array along the second axis.

Input Format:

- A 2D array arr of shape (m, n) where m is the number of rows and n is the number of columns. This array is already provided.

Output Format:

- Print the diff array, which represents the difference between the maximum and minimum values along the second axis (rows) of the array.

Source Code:

MaxMinDiff.py

```
import numpy as np

# Sample array
arr = np.array([[4, 5, 2],
                [9, 3, 7],
                [1, 6, 8]])

# Write code below to achieve the goal of this question
r=[]
for i in arr:
    r.append(max(i)-min(i))
print(np.array(r))
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
[3 6 7]

S.No: 30	Exp. Name: <i>NumPy and SciPy</i>	Date: 2025-05-05
-----------------	--	-------------------------

Aim:

Write a Python program to create a 2-D NumPy array with ones on the diagonal and zeros elsewhere. Convert this array into a SciPy sparse matrix in CSR (Compressed Sparse Row) format and display both the original array and the sparse matrix.

Input Format:

- An integer size representing the size of the 2-D array (e.g., size = 5 for a 5x5 matrix).

Output Format:

- Print the original 2-D NumPy array with ones on the diagonal and zeros elsewhere.
- Print the sparse matrix in CSR format. Restructured the text

Source Code:

NumPySciPy.py

```

"""import numpy as np
from scipy.sparse import csr_matrix

# Create a 2-D array with ones on the diagonal and zeros
elsewhere
size = 5
arr = np.eye(size)

# Write code below to convert the NumPy array to a SciPy sparse
matrix in CSR format
"""
import numpy as np
from scipy.sparse import csr_matrix
def create_sparse_matrix(size):
    array = np.eye(size)
    sparse_matrix = csr_matrix(array)
    print("Original array:")
    print(array)
    print("Sparse matrix in CSR format:")
    print(sparse_matrix)

size = int(input("Enter the size of the matrix: "))
create_sparse_matrix(size)

```


Execution Results - All test cases have succeeded!

Test Case - 1	
User Output	
Enter the size of the matrix:	
3	
Original array:	
[[1. 0. 0.]	
[0. 1. 0.]	
[0. 0. 1.]]	
Sparse matrix in CSR format:	
(0, 0)	1.0
(1, 1)	1.0
(2, 2)	1.0

Test Case - 2	
User Output	
Enter the size of the matrix:	
2	
Original array:	
[[1. 0.]	
[0. 1.]]	
Sparse matrix in CSR format:	
(0, 0)	1.0
(1, 1)	1.0

Page No: 73

ID: 2415249

2024-28-CSE1

Guru Nanak Dev Engineering College

S.No: 31	Exp. Name: Pandas	Date: 2025-05-05
-----------------	--------------------------	-------------------------

Aim:

Write a Python program to add, subtract, multiply, and divide two Pandas Series element-wise.

Input Format:

- Two Pandas Series s1 and s2 with the same length. This is already provided.

Output Format:

A new Pandas Series for each operation:

- Addition: A Series with the element-wise sum.
- Subtraction: A Series with the element-wise difference.
- Multiplication: A Series with the element-wise product.
- Division: A Series with the element-wise division.

Source Code:

PandasSeries.py

```
import pandas as pd

# Create two Pandas Series
s1 = pd.Series([1, 2, 3, 4, 5])
s2 = pd.Series([6, 7, 8, 9, 10])

# Write code below to perform addition
print("Addition:")
print(pd.Series(s1+s2))

# Write code below to perform subtraction
print('Subtraction:')
print(pd.Series(s1-s2))
# Write code below to perform multiplication
print('Multiplication:')
print(pd.Series(s1*s2))
# Write code below to perform division
print("Division:")
print(pd.Series(s1/s2))
```

Execution Results - All test cases have succeeded!

Test Case - 1	
User Output	
Addition:	
0	7
1	9
2	11
3	13
4	15
dtype: int64	
Subtraction:	
0	-5
1	-5
2	-5
3	-5
4	-5
dtype: int64	
Multiplication:	
0	6
1	14
2	24
3	36
4	50
dtype: int64	
Division:	
0	0.166667
1	0.285714
2	0.375000
3	0.444444
4	0.500000
dtype: float64	

S.No: 32	Exp. Name: Pandas / Matplotlib	Date: 2025-05-05
-----------------	---------------------------------------	-------------------------

Aim:

Write a Pandas program to create a line plot using matplotlib. The line plot must be of the opening, closing stock prices of Alphabet Inc. between the dates 2020-04-01 and 2020-10-31, both inclusive. Please use the data available in the file alphabet_stock_data.csv for this purpose.

Note: Take figure size as (12,6).

Source Code:

PandasMatplotlib.py

```
import pandas as pd
import matplotlib.pyplot as plt
from pyodide.http import open_url

url = 'https://ct-public-bucket.s3.ap-south-1.amazonaws.com/lms/niet/alphabet_stock_data.csv'

csv_data = open_url(url)
df = pd.read_csv(csv_data)
df['Date']=pd.to_datetime(df['Date'])
start_date='2020-04-01'
end_date='2020-10-31'
df_filtered=df[(df['Date']>=start_date) & (df['Date']<=end_date)]
plt.figure(figsize=(12,6))
plt.plot(df_filtered['Date'],df_filtered['Open'],label='Opening Price')
plt.plot(df_filtered['Date'],df_filtered['Close'],label='Closing price')
plt.title('Alphabet Inc. Stock Prices')
plt.xlabel('Date')
plt.ylabel('Price')
plt.legend(loc='upper right')
plt.xticks(rotation=30,ha='right')
plt.subplots_adjust(bottom=0.2)
plt.show()
```

Execution Results - All test cases have succeeded!

Test Case - 1

Page No: 76

ID: 2415249

2024-28-CSE1

Guru Nanak Dev Engineering College

User Output
data:img/png;base64,iVBORw0KGgoAAAANSUhEUgAABLAAAAJYCAYAABy5h8a